Assignment 7

Data Loading, Storage and File Formats Problem Statement: Analyzing Sales Data from Multiple File Formats Dataset: Sales data in multiple file formats (e.g., CSV, Excel, JSON) Description: The goal is to load and analyze sales data from different file formats, including CSV, Excel, and JSON, and perform data cleaning, transformation, and analysis on the dataset. Tasks to Perform: Obtain sales data files in various formats, such as CSV, Excel, and JSON. 1. Load the sales data from each file format into the appropriate data structures or dataframes. 2. Explore the structure and content of the loaded data, identifying any inconsistencies, missing values, or data quality issues. 3. Perform data cleaning operations, such as handling missing values, removing duplicates, or correcting inconsistencies. 4. Convert the data into a unified format, such as a common dataframe or data structure, to enable seamless analysis. 5. Perform data transformation tasks, such as merging multiple datasets, splitting columns, or deriving new variables. 6. Analyze the sales data by performing descriptive statistics, aggregating data by specific variables, or calculating metrics such as total sales, average order value, or product category distribution. 7. Create visualizations, such as bar plots, pie charts, or box plots, to represent the sales data and gain insights into sales trends, customer behavior, or product performance.

Problem Statement

Analyzing Sales Data from Multiple File Formats

Dataset

Sales data in CSV, Excel and JSON formats (for example: Sales_Data.csv, Sales_Data.xlsx and Sales_Data.json).

1. Aim

To load sales data from different file formats such as CSV, Excel and JSON, explore and clean the data, convert it into a unified format, perform transformation and statistical analysis, and create suitable visualizations.

2. Theory

Data is often stored in different file formats depending on the source. CSV (Comma-Separated Values) stores tabular data in rows and columns. Excel files can contain structured data in worksheets, while JSON stores data using key-value structures.

Pandas is a Python library widely used for data loading and analysis. The functions read_csv(), read_excel() and read_json() allow data to be imported into DataFrames. A DataFrame is a two-dimensional labelled data structure containing rows and columns.

After loading data, it is important to inspect its structure using head(), info(), columns and describe(). Data quality can be checked using isnull() for missing values and duplicated() for duplicate records. Cleaning improves the reliability of analysis.

When information from different files represents the same type of data, it can be converted into a common DataFrame. Pandas concat() can combine compatible datasets. groupby() is then used to aggregate data, such as calculating total sales by product category.

Visualization helps communicate the results. Bar charts can compare category-wise sales, pie charts can show distribution, and box plots can show the spread of sales values.

3. Objectives

- Load data from CSV, Excel and JSON files.
- Explore the structure, columns, data types and statistical summary.
- Identify missing values, duplicates and inconsistencies.
- Clean the datasets using suitable methods.
- Convert data into a unified DataFrame.
- Perform data transformation and aggregation.
- Calculate total sales, average sales and category distribution.
- Represent findings using appropriate plots.

4. Software / Hardware Requirements

- Python 3.x
- Jupyter Notebook / Google Colab / any Python IDE
- Pandas
- Matplotlib
- Excel reader support such as openpyxl when required

5. Steps / Procedure

1. Import Pandas and Matplotlib.
2. Obtain the sales files in CSV, Excel and JSON formats.
3. Load each file using the appropriate Pandas function.
4. Display the first few records and inspect columns and data types.
5. Check for missing values and duplicate records.
6. Remove duplicate records and handle missing values appropriately.
7. Convert the datasets into a common DataFrame and combine compatible records.
8. Perform descriptive statistical analysis.
9. Group sales according to product category and calculate total sales.
10. Create bar, pie or box plots.
11. Observe the results and record important findings.

6. Algorithm

1. Start.
2. Import required Python libraries.
3. Read the CSV, Excel and JSON datasets.
4. Display the loaded datasets and inspect their structure.
5. Check missing values and duplicate records.
6. Clean the data.
7. Combine compatible datasets into one DataFrame.
8. Calculate descriptive statistics.
9. Group the sales data by category and calculate total sales.
10. Generate suitable visualizations.
11. Display and interpret the results.
12. Stop.

7. Python Program

```
import pandas as pd
import matplotlib.pyplot as plt

# 1. Load data from different file formats
csv_data = pd.read_csv("Sales_Data.csv")
excel_data = pd.read_excel("Sales_Data.xlsx")
```

```
json_data = pd.read_json("Sales_Data.json")

# 2. Display sample records
print("CSV Data:")
print(csv_data.head())

print("\nExcel Data:")
print(excel_data.head())

print("\nJSON Data:")
print(json_data.head())

# 3. Explore the CSV dataset
print("\nDataset Information:")
csv_data.info()

print("\nMissing Values:")
print(csv_data.isnull().sum())

print("\nDuplicate Records:")
print(csv_data.duplicated().sum())

# 4. Remove duplicate records
csv_data = csv_data.drop_duplicates()

# 5. Example of missing-value handling
if "Sales" in csv_data.columns:
    csv_data["Sales"] = csv_data["Sales"].fillna(
        csv_data["Sales"].mean()
    )

# 6. Combine compatible datasets
combined_data = pd.concat(
    [csv_data, excel_data, json_data],
    ignore_index=True
)

print("\nCombined Data:")
print(combined_data.head())

# 7. Statistical summary
print("\nStatistical Summary:")
print(combined_data.describe())

# 8. Category-wise sales aggregation
if "Category" in combined_data.columns and "Sales" in combined_data.columns:
    category_sales = combined_data.groupby(
        "Category"
    )["Sales"].sum()

    print("\nTotal Sales by Category:")
    print(category_sales)

    # 9. Bar chart
    category_sales.plot(
        kind="bar",
        figsize=(8, 5)
    )
```

```
plt.title("Sales by Product Category")
plt.xlabel("Category")
plt.ylabel("Total Sales")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# 10. Pie chart
category_sales.plot(
    kind="pie",
    autopct="%1.1f%%",
    figsize=(7, 7)
)

plt.title("Product Category Distribution")
plt.ylabel("")
plt.show()
```

8. Explanation of Important Statements

- read_csv() loads the CSV file into a DataFrame.
- read_excel() reads an Excel worksheet.
- read_json() loads JSON records into a DataFrame.
- head() displays the first five records.
- info() provides column, data-type and non-null information.
- isnull().sum() counts missing values in each column.
- duplicated().sum() counts duplicate rows.
- drop_duplicates() removes duplicate records.
- fillna() replaces missing values with a selected value such as the mean.
- concat() combines DataFrames vertically.
- describe() generates descriptive statistics.
- groupby() groups records according to a category and sum() calculates total sales.
- plot() is used to create graphical representations.

9. Expected Output / Observation

The student should record the number of records, columns, missing values and duplicates observed in the dataset. The category-wise sales table and generated charts should be attached or drawn in the practical journal.

10. Result

Sales data from CSV, Excel and JSON formats was successfully loaded, explored, cleaned, combined, analyzed and visualized.

11. Conclusion

The practical demonstrates how data stored in different file formats can be brought into a common analytical environment and converted into useful sales insights.

Assignment 9

Data Cleaning and Preparation Problem Statement: Analyzing Customer Churn in a Telecommunications Company Dataset: "Telecom_Customer_Churn.csv" Description: The dataset contains information about customers of a telecommunications company and whether they have churned (i.e., discontinued their services). The dataset includes various attributes of the customers, such as their demographics, usage patterns, and account information. The goal is to perform data cleaning and preparation to gain insights into the factors that contribute to customer churn. Tasks to Perform: 1. Import the "Telecom_Customer_Churn.csv" dataset. 2. Explore the dataset to understand its structure and content. 3. Handle missing values in the dataset, deciding on an appropriate strategy. 4. Remove any duplicate records from the dataset. 5. Check for inconsistent data, such as inconsistent formatting or spelling variations, and standardize it. 6. Convert columns to the correct data types as needed. 7. Identify and handle outliers in the data. 8. Perform feature engineering, creating new features that may be relevant to predicting customer churn. 9. Normalize or scale the data if necessary. 10. Split the dataset into training and testing sets for further analysis. 11. Export the cleaned dataset for future analysis or modeling.

Problem Statement

Analyzing Customer Churn in a Telecommunications Company

Dataset

Telecom_Customer_Churn.csv

1. Aim

To clean and prepare a telecommunications customer churn dataset by handling missing values, duplicates, inconsistent data, data types and outliers, and prepare the data for further analysis or modeling.

2. Theory

Data cleaning is an important stage of the data-analysis process. Real-world datasets may contain missing values, duplicate records, inconsistent spellings, incorrect data types and extreme observations. Missing values can be handled by deleting records when appropriate or by replacing values using methods such as mean, median or mode. The method should depend on the nature of the variable. Duplicate records can produce biased results and should be identified and removed when they represent repeated observations. Categorical values may need standardization so that variations in spelling or capitalization are treated consistently. Correct data types are important for analysis. For example, a numeric column stored as text must be converted to a numerical type before mathematical operations are performed. Outliers are unusually large or small observations. They can be identified using box plots, IQR-based methods or domain knowledge. Feature engineering creates new variables that may be useful for analysis or prediction. Numerical variables may also be normalized or standardized before machine-learning algorithms are applied.

3. Objectives

- Import the Telecom_Customer_Churn.csv dataset.
- Understand its structure and content.
- Handle missing values appropriately.
- Remove duplicate records.
- Standardize inconsistent data.
- Convert columns to correct data types.
- Identify and handle outliers.
- Perform feature engineering when required.

- Normalize or scale numerical features when required.
- Prepare the data for training and testing.
- Export the cleaned dataset.

4. Software / Hardware Requirements

- Python
- Pandas
- NumPy when required
- Matplotlib
- Scikit-learn when scaling or train-test splitting is required

5. Steps / Procedure

1. Import the dataset using Pandas.
2. Display sample records and dataset information.
3. Check missing values.
4. Check and remove duplicate records.
5. Standardize inconsistent categorical values.
6. Convert columns to correct data types.
7. Inspect numerical variables for outliers.
8. Handle outliers according to the analysis requirement.
9. Create useful derived features if required.
10. Normalize or scale numerical variables if required.
11. Split into training and testing data if modeling is planned.
12. Export the cleaned dataset.

6. Algorithm

1. Start.
2. Read the CSV file.
3. Display its shape, columns, data types and summary statistics.
4. Check missing values and duplicates.
5. Handle missing values.
6. Remove duplicate records.
7. Standardize inconsistent values.
8. Convert columns to appropriate types.
9. Detect and handle outliers.
10. Perform feature engineering if required.
11. Scale numerical features if required.
12. Save the cleaned dataset.
13. Stop.

7. Python Program

```
import pandas as pd

# Load dataset
df = pd.read_csv("Telecom_Customer_Churn.csv")

# Explore dataset
print(df.head())
print("\nShape:", df.shape)
print("\nColumns:", df.columns.tolist())
print("\nData Types:")
print(df.dtypes)
```

```
print("\nStatistical Summary:")
print(df.describe())

# Missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Duplicate records
print("\nDuplicate Records:", df.duplicated().sum())
df = df.drop_duplicates()

# Convert numeric column if it exists
if "TotalCharges" in df.columns:
    df["TotalCharges"] = pd.to_numeric(
        df["TotalCharges"],
        errors="coerce"
    )

# Fill numerical missing values with median
numeric_columns = df.select_dtypes(
    include="number"
).columns

for col in numeric_columns:
    df[col] = df[col].fillna(df[col].median())

print("\nMissing Values After Cleaning:")
print(df.isnull().sum())

# Export
df.to_csv(
    "Cleaned_Telecom_Customer_Churn.csv",
    index=False
)

print("\nCleaned dataset saved successfully.")
```

8. Explanation of Important Statements

- shape gives rows and columns.
- dtypes identifies the current data types.
- isnull().sum() counts missing values.
- duplicated() identifies repeated records.
- to_numeric() converts a column to numeric form and can turn invalid values into missing values.
- median() is less affected by extreme values than the mean and can be useful for filling numerical missing values.
- to_csv() exports the cleaned dataset.

9. Expected Output / Observation

Record the number of rows and columns before and after cleaning, the missing-value counts and the number of duplicate records removed. Note any important data-quality issues.

10. Result

The customer churn dataset was successfully explored, cleaned and exported for further analysis.

11. Conclusion

Proper data cleaning and preparation improves data quality and makes the dataset more suitable for statistical analysis and machine-learning tasks.

Assignment 10

Data Wrangling Problem Statement: Data Wrangling on Real Estate Market Dataset: "RealEstate_Prices.csv" Description: The dataset contains information about housing prices in a specific real estate market. It includes various attributes such as property characteristics, location, sale prices, and other relevant features. The goal is to perform data wrangling to gain insights into the factors influencing housing prices and prepare the dataset for further analysis or modeling. Tasks to Perform: 1. Import the "RealEstate_Prices.csv" dataset. Clean column names by removing spaces, special characters, or renaming them for clarity. 2. Handle missing values in the dataset, deciding on an appropriate strategy (e.g., imputation or removal). 3. Perform data merging if additional datasets with relevant information are available (e.g., neighborhood demographics or nearby amenities). 4. Filter and subset the data based on specific criteria, such as a particular time period, property type, or location. 5. Handle categorical variables by encoding them appropriately (e.g., one-hot encoding or label encoding) for further analysis. 6. Aggregate the data to calculate summary statistics or derived metrics such as average sale prices by neighborhood or property type. 7. Identify and handle outliers or extreme values in the data that may affect the analysis or modeling process.

Problem Statement

Data Wrangling on Real Estate Market

Dataset

RealEstate_Prices.csv

1. Aim

To perform data wrangling on real-estate data by cleaning column names, handling missing values, filtering records, encoding categorical variables, aggregating data and handling outliers.

2. Theory

Data wrangling, also called data munging, is the process of transforming raw data into a structured and usable form. It is usually performed before analysis or model building.

A real-estate dataset may contain property characteristics, locations, property types and sale prices. Column names may contain spaces or special characters, which can make programming inconvenient. Standardizing names improves readability.

Filtering allows analysts to select records satisfying particular conditions, such as a specific property type, location or time period. Categorical variables can be converted into numerical representations using label encoding or one-hot encoding.

Aggregation summarizes data. For example, the average sale price can be calculated for each neighborhood or property type. Outliers can be identified because extremely high or low prices may influence averages and machine-learning models.

3. Objectives

- Import the real-estate dataset.
- Clean and standardize column names.
- Handle missing values.
- Merge relevant datasets when available.
- Filter and subset records.
- Encode categorical variables.
- Calculate summary statistics.
- Identify and handle extreme values.

4. Software / Hardware Requirements

- Python
- Pandas
- NumPy
- Scikit-learn if categorical encoding is required

5. Steps / Procedure

1. Load the CSV file.
2. Inspect rows, columns and data types.
3. Standardize column names.
4. Check missing values.
5. Remove or impute missing values as appropriate.
6. Remove duplicates.
7. Filter records based on selected conditions.
8. Encode categorical variables when required.
9. Group data and calculate average sale prices.
10. Inspect numerical columns for outliers.
11. Prepare the final wrangled dataset.

6. Algorithm

1. Start.
2. Read RealEstate_Prices.csv.
3. Inspect the dataset.
4. Clean column names.
5. Check and handle missing values.
6. Remove duplicate records.
7. Filter the required subset.
8. Encode categorical variables if needed.
9. Group records and calculate summary statistics.
10. Identify and handle outliers.
11. Display the final dataset.
12. Stop.

7. Python Program

```
import pandas as pd

# Load data
df = pd.read_csv("RealEstate_Prices.csv")

print("Original Data:")
print(df.head())
```

```
# Clean column names
df.columns = (
    df.columns
    .str.strip()
    .str.lower()
    .str.replace(" ", "_")
    .str.replace(r"[^a-zA-Z0-9_]", "", regex=True)
)

print("\nCleaned Column Names:")
print(df.columns.tolist())

# Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Remove duplicate records
df = df.drop_duplicates()

# Example: filter properties if columns exist
if "sale_price" in df.columns:
    filtered = df[df["sale_price"] > df["sale_price"].median()]
    print("\nProperties above median price:")
    print(filtered.head())

# Aggregation
if "property_type" in df.columns and "sale_price" in df.columns:
    average_price = df.groupby(
        "property_type"
    )["sale_price"].mean()

    print("\nAverage Sale Price by Property Type:")
    print(average_price)

print("\nFinal Data:")
print(df.head())
```

8. Explanation of Important Statements

- strip() removes unwanted spaces from column names.
- lower() converts names to lowercase.
- replace() standardizes spaces and special characters.
- Filtering selects records that satisfy a condition.
- groupby() groups records by property type.
- mean() calculates the average sale price.

9. Expected Output / Observation

Record the cleaned column names, missing values, number of duplicates removed and the average sale price for each selected category.

10. Result

The real-estate dataset was successfully cleaned, filtered and aggregated.

11. Conclusion

Data wrangling converts raw real-estate data into a cleaner and more structured form suitable for analysis and modeling.

Assignment 11

Data Visualization using matplotlib Problem Statement: Analyzing Air Quality Index (AQI) Trends in a City Dataset: "City_Air_Quality.csv" Description: The dataset contains information about air quality measurements in a specific city over a period of time. It includes attributes such as date, time, pollutant levels (e.g., PM2.5, PM10, CO), and the Air Quality Index (AQI) values. The goal is to use the matplotlib library to create visualizations that effectively represent the AQI trends and patterns for different pollutants in the city. Tasks to Perform: 1. Import the "City_Air_Quality.csv" dataset. 2. Explore the dataset to understand its structure and content. 3. Identify the relevant variables for visualizing AQI trends, such as date, pollutant levels, and AQI values. 4. Create line plots or time series plots to visualize the overall AQI trend over time. 5. Plot individual pollutant levels (e.g., PM2.5, PM10, CO) on separate line plots to visualize their trends over time. 6. Use bar plots or stacked bar plots to compare the AQI values across different dates or time periods. 7. Create box plots or violin plots to analyze the distribution of AQI values for different pollutant categories. 8. Use scatter plots or bubble charts to explore the relationship between AQI values and pollutant levels. 9. Customize the visualizations by adding labels, titles, legends, and appropriate color schemes.

Problem Statement

Analyzing Air Quality Index (AQI) Trends in a City

Dataset

City_Air_Quality.csv

1. Aim

To use Matplotlib to create visualizations that represent AQI trends and pollutant patterns such as PM2.5, PM10 and CO over time.

2. Theory

Data visualization presents numerical information graphically so that trends, comparisons, distributions and relationships can be understood easily.

Matplotlib is a Python visualization library. A line plot is useful for time-series trends. Bar charts compare values across categories or dates. Box plots show the distribution and possible outliers. Scatter plots show relationships between two numerical variables.

In an AQI dataset, date and time are important for studying changes over time. Pollutant variables such as PM2.5 and PM10 can be plotted against AQI to investigate whether pollutant levels are associated with changes in air quality.

A good visualization should contain an informative title, axis labels, suitable tick labels and legends where necessary.

3. Objectives

- Import and inspect the AQI dataset.
- Identify date, pollutant and AQI variables.
- Create an AQI time-series plot.
- Visualize individual pollutant trends.
- Compare AQI values using bar plots.
- Study distributions using box plots.

- Study relationships using scatter plots.
- Add appropriate titles, labels and legends.

4. Software / Hardware Requirements

- Python
- Pandas
- Matplotlib
- City_Air_Quality.csv

5. Steps / Procedure

1. Load the dataset.
2. Explore the columns and data types.
3. Convert the date column if required.
4. Create a line plot of AQI over time.
5. Create separate plots for PM2.5, PM10 and CO.
6. Create a bar chart for comparison.
7. Create a box plot to examine distribution.
8. Create a scatter plot of pollutant level versus AQI.
9. Customize the charts.
10. Interpret the visual results.

6. Algorithm

1. Start.
2. Import Pandas and Matplotlib.
3. Read City_Air_Quality.csv.
4. Inspect the dataset.
5. Identify date, pollutant and AQI columns.
6. Plot AQI against date.
7. Plot pollutant values against date.
8. Create comparison and distribution plots.
9. Create a scatter plot to study relationships.
10. Add titles and labels.
11. Display the plots.
12. Stop.

7. Python Program

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("City_Air_Quality.csv")

print(df.head())
print(df.info())

# AQI trend
plt.figure(figsize=(10,5))
plt.plot(df["Date"], df["AQI"], marker="o")
plt.title("AQI Trend Over Time")
plt.xlabel("Date")
plt.ylabel("AQI")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
# PM2.5 trend
plt.figure(figsize=(10,5))
plt.plot(df["Date"], df["PM2.5"], marker="o")
plt.title("PM2.5 Trend Over Time")
plt.xlabel("Date")
plt.ylabel("PM2.5")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Scatter plot
plt.figure(figsize=(7,5))
plt.scatter(df["PM2.5"], df["AQI"])
plt.title("Relationship Between PM2.5 and AQI")
plt.xlabel("PM2.5")
plt.ylabel("AQI")
plt.tight_layout()
plt.show()

# Box plot
plt.figure(figsize=(6,5))
plt.boxplot(df["AQI"].dropna())
plt.title("Distribution of AQI")
plt.ylabel("AQI")
plt.show()
```

8. Explanation of Important Statements

- plot() creates a line graph.
- scatter() creates a scatter plot.
- boxplot() shows the distribution and possible outliers.
- xlabel() and ylabel() label the axes.
- title() gives the graph a descriptive title.
- xticks(rotation=45) improves readability of date labels.
- tight_layout() reduces overlapping elements.

9. Expected Output / Observation

Record the overall AQI trend, highest and lowest AQI values if required, pollutant trends and the relationship visible in the scatter plot.

10. Result

AQI and pollutant trends were successfully visualized using line, scatter and box plots.

11. Conclusion

Matplotlib provides effective graphical methods for understanding air-quality trends, pollutant distributions and relationships.

Assignment 12

Data Aggregation Problem Statement: Analyzing Sales Performance by Region in a Retail Company Dataset: "Retail_Sales_Data.csv" Description: The dataset contains information about sales transactions in a retail company. It includes attributes such as transaction date, product category, quantity sold, and sales amount. The goal is to perform data aggregation to analyze the sales performance by region and identify the top-performing regions. Tasks to Perform: 1. Import the "Retail_Sales_Data.csv" dataset. 2. Explore the dataset to understand its structure and content. 3. Identify the relevant variables for aggregating sales data, such as region, sales amount, and product category. 4. Group the sales data by region and calculate the total sales amount for each region. 5. Create bar plots or pie charts to visualize the sales distribution by region. 6. Identify the top-performing regions based on the highest sales amount. 7. Group the sales data by region and product category to calculate the total sales amount for each combination. 8. Create stacked bar plots or grouped bar plots to compare the sales amounts across different regions and product categories.

Problem Statement

Analyzing Sales Performance by Region in a Retail Company

Dataset

Retail_Sales_Data.csv

1. Aim

To aggregate retail sales data by region and product category, calculate total sales and identify the top-performing region.

2. Theory

Data aggregation summarizes a large number of transaction records into meaningful statistics. Instead of examining every transaction individually, records can be grouped by a variable such as region or product category.

Pandas groupby() is one of the most important functions for aggregation. After grouping, functions such as sum(), mean(), count(), min() and max() can be applied.

In this practical, sales transactions are grouped by region to calculate total sales. The region with the highest total sales is identified as the top-performing region. A second grouping by region and product category allows comparison of product performance across regions.

Bar charts and pie charts can communicate the distribution of sales, while grouped or stacked bar charts can compare multiple categories.

3. Objectives

- Import and explore the retail sales dataset.
- Identify region, sales amount and product category.
- Calculate total sales by region.
- Visualize regional sales.
- Identify the top-performing region.
- Calculate sales by region and product category.
- Compare sales across regions and categories.

4. Software / Hardware Requirements

- Python
- Pandas
- Matplotlib
- Retail_Sales_Data.csv

5. Steps / Procedure

1. Load the CSV dataset.
2. Inspect its structure.
3. Identify relevant columns.
4. Group transactions by Region.
5. Calculate total sales using sum().
6. Find the region with maximum sales.
7. Group data by Region and Product_Category.
8. Create bar or pie charts.
9. Create grouped/stacked charts if required.
10. Interpret the results.

6. Algorithm

1. Start.
2. Import required libraries.
3. Read the retail sales CSV file.
4. Inspect the dataset.
5. Group records by Region.
6. Calculate total sales for every region.
7. Find the region with maximum sales.
8. Group records by Region and Product_Category.
9. Calculate total sales for each combination.
10. Create visualizations.
11. Display results.
12. Stop.

7. Python Program

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("Retail_Sales_Data.csv")

print("First five records:")
print(df.head())

print("\nDataset Information:")
df.info()

# Total sales by region
region_sales = df.groupby(
    "Region"
)["Sales_Amount"].sum()

print("\nTotal Sales by Region:")
print(region_sales)

# Top-performing region
```

```
top_region = region_sales.idxmax()
top_sales = region_sales.max()

print("\nTop Performing Region:", top_region)
print("Highest Sales:", top_sales)

# Bar chart
region_sales.plot(
    kind="bar",
    figsize=(8,5)
)

plt.title("Sales Distribution by Region")
plt.xlabel("Region")
plt.ylabel("Total Sales")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Region and product category
region_category = df.groupby(
    ["Region", "Product_Category"]
)["Sales_Amount"].sum()

print("\nSales by Region and Product Category:")
print(region_category)
```

8. Explanation of Important Statements

- groupby('Region') creates one group for each region.
- sum() adds sales amounts within each group.
- idxmax() returns the label corresponding to the maximum value.
- max() returns the maximum sales amount.
- Grouping by two columns creates a multi-level grouping for regional and category analysis.

9. Expected Output / Observation

Record total sales for each region and identify the region with the highest sales. Also record the regional product-category totals.

10. Result

Retail sales were successfully aggregated by region and product category, and the top-performing region was identified.

11. Conclusion

Data aggregation makes it easier to summarize transaction-level data and compare the performance of different regions and product categories.

Assignment 13

Time Series Data Analysis Problem statement: Analysis and Visualization of Stock Market Data Dataset: "Stock_Prices.csv" Description: The dataset contains historical stock price data for a particular company over a period of time. It includes attributes such as date, closing price, volume, and other relevant features. The goal is to perform time series data analysis on the stock price data to identify trends, patterns, and potential predictors, as well as build models to forecast future stock prices. Tasks to Perform: 1. Import the "Stock_Prices.csv" dataset. 2. Explore the dataset to understand its structure and content. 3. Ensure that the date column is in the appropriate format (e.g., datetime) for time series analysis. 4. Plot line charts or time series plots to visualize the historical stock price trends over time. 5. Calculate and plot moving averages or rolling averages to identify the underlying trends and smooth out noise. 6. Perform seasonality analysis to identify periodic patterns in the stock prices, such as weekly, monthly, or yearly fluctuations. 7. Analyze and plot the correlation between the stock prices and other variables, such as trading volume or market indices. 8. Use autoregressive integrated moving average (ARIMA) models or exponential smoothing models to forecast future stock prices.

Problem Statement

Analysis and Visualization of Stock Market Data

Dataset

Stock_Prices.csv

1. Aim

To analyze historical stock-price data, identify trends and patterns, calculate moving averages, study correlations and prepare the data for forecasting.

2. Theory

Time series data consists of observations recorded in chronological order. Stock-market data is a common example because prices and trading volumes are recorded according to dates.

For time-series analysis, the date column should be converted into a datetime format and the records should be sorted chronologically. A line chart can then show how the closing price changes over time.

A moving average is calculated by taking the average of a fixed number of recent observations. It smooths short-term fluctuations and helps identify the underlying trend.

Seasonality analysis can be used to investigate recurring patterns over weekly, monthly or yearly periods. Correlation analysis can examine whether stock price is related to variables such as trading volume or market indices.

Forecasting methods such as ARIMA or exponential smoothing can be applied when appropriate. Forecasting requires suitable time-series preparation and model evaluation.

3. Objectives

- Import the stock-price dataset.
- Explore its structure and content.
- Convert the date column to datetime format.
- Plot historical stock-price trends.

- Calculate and plot moving averages.
- Investigate periodic patterns when sufficient data is available.
- Analyze correlations between stock prices and other variables.
- Prepare data for forecasting using ARIMA or exponential smoothing when required.

4. Software / Hardware Requirements

- Python
- Pandas
- Matplotlib
- Stock_Prices.csv
- Statsmodels may be used for ARIMA/exponential smoothing

5. Steps / Procedure

1. Load the stock dataset.
2. Inspect the columns and data types.
3. Convert Date to datetime.
4. Sort records by Date.
5. Plot Closing_Price against Date.
6. Calculate a rolling/moving average.
7. Plot closing price and moving average together.
8. Calculate correlations with variables such as Volume.
9. Study seasonality when sufficient historical data is available.
10. Apply an appropriate forecasting model when required.
11. Interpret the results.

6. Algorithm

1. Start.
2. Import Pandas and Matplotlib.
3. Read Stock_Prices.csv.
4. Convert Date into datetime format.
5. Sort records by Date.
6. Plot historical closing prices.
7. Calculate the moving average using a rolling window.
8. Plot the moving average with closing price.
9. Calculate correlation between selected numerical variables.
10. Perform seasonality or forecasting analysis if required.
11. Display and interpret results.
12. Stop.

7. Python Program

```
import pandas as pd
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv("Stock_Prices.csv")

# Convert Date to datetime
df["Date"] = pd.to_datetime(df["Date"])

# Sort chronologically
df = df.sort_values("Date")
```

```
print(df.head())
print(df.info())

# Historical stock price
plt.figure(figsize=(10,5))
plt.plot(df["Date"], df["Closing_Price"])
plt.title("Historical Stock Price")
plt.xlabel("Date")
plt.ylabel("Closing Price")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Seven-day moving average
df["Moving_Average"] = (
    df["Closing_Price"]
    .rolling(window=7)
    .mean()
)

plt.figure(figsize=(10,5))
plt.plot(
    df["Date"],
    df["Closing_Price"],
    label="Closing Price"
)

plt.plot(
    df["Date"],
    df["Moving_Average"],
    label="7-Day Moving Average"
)

plt.title("Stock Price and Moving Average")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Correlation
print("\nCorrelation:")
print(
    df[["Closing_Price", "Volume"]].corr()
)
```

8. Explanation of Important Statements

- to_datetime() converts text dates into datetime values.
- sort_values() arranges observations chronologically.
- plot() displays the historical price trend.
- rolling(window=7).mean() calculates a seven-observation moving average.
- A moving average smooths short-term variation.
- corr() calculates correlation between numerical variables.

9. Expected Output / Observation

Record the overall stock-price trend, important increases/decreases, the behavior of the moving average and the correlation between closing price and volume.

10. Result

Historical stock prices were successfully converted to time-series data, visualized, smoothed using a moving average and correlated with trading volume.

11. Conclusion

Time-series analysis provides useful methods for understanding stock-price trends, smoothing fluctuations and preparing historical data for forecasting.